



CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

NÍVEA PINTO DE SOUZA

1- Explique com suas palavras a diferença entre backend, API e banco de dados.

O backend é a parte do sistema responsável por receber informações, aplicar regras de negócio, validar dados e coordenar o que deve acontecer dentro da aplicação. É nele, por exemplo, que podemos impedir que um horário já ocupado seja agendado novamente. A API é uma forma de comunicação com o backend. Ela funciona como uma porta de entrada pela qual outros sistemas ou aplicações podem enviar solicitações e receber respostas. Portanto, a API não é o backend inteiro, mas uma maneira de acessar as funcionalidades que ele disponibiliza. O próprio material reforça que a API é uma interface de comunicação, enquanto o backend reúne as responsabilidades que existem por trás dela. Já o banco de dados é responsável principalmente por armazenar e consultar as informações do sistema. Ele pode guardar, por exemplo, clientes, serviços e agendamentos. Porém, quem determina como essas informações podem ser utilizadas e quais regras precisam ser respeitadas é o backend.

2- Regras que devem ser garantidas no backend

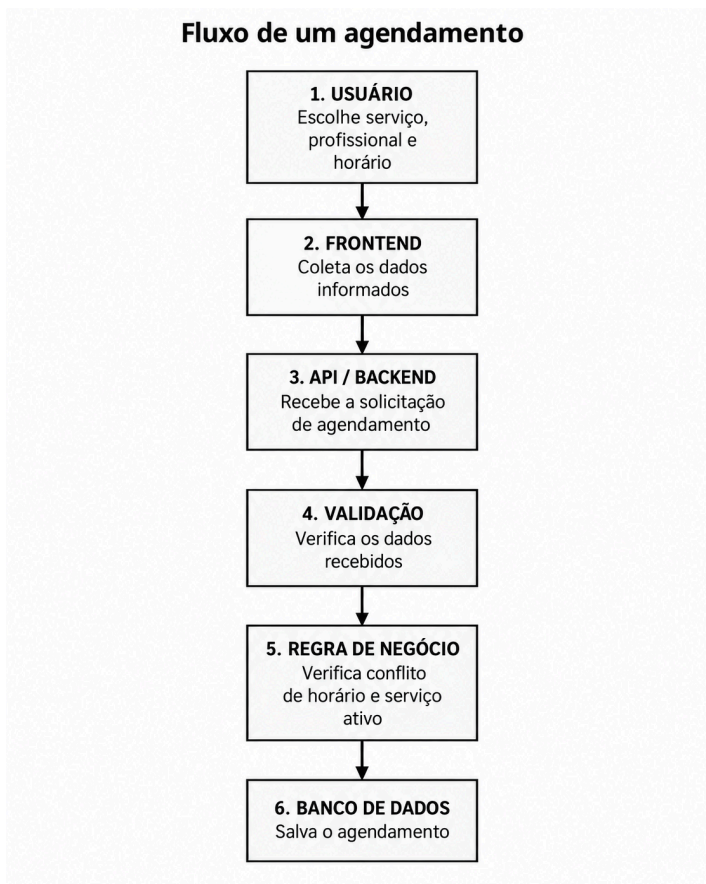
No domínio de agendamentos, o backend deve garantir que um profissional não possua dois agendamentos no mesmo horário, que serviços desativados não possam ser agendados e que todo agendamento esteja relacionado a um profissional e a um serviço válido. Essas regras devem ser verificadas no backend para que não dependam apenas da interface utilizada pelo usuário.

3- Validação sintática x regra de negócio

O e-mail sem @, o preço negativo e o CPF com quantidade incorreta de dígitos podem ser classificados como validações sintáticas, pois verificam se os dados recebidos estão dentro de um formato ou condição aceitável. Já o conflito de horário e o agendamento de um serviço desativado são regras de negócio, pois dependem das regras e do estado da aplicação para determinar se determinada ação pode ou não ser realizada.

4- Diagrama

Em um sistema de agendamento online, o processo começa quando o usuário escolhe um serviço, um profissional e um horário. O frontend coleta essas informações e envia a solicitação para o backend. O backend realiza as validações necessárias e aplica as regras de negócio, como verificar se o serviço está ativo e se o profissional já possui outro agendamento no mesmo horário. Caso todas as condições sejam atendidas, o agendamento é salvo no banco de dados.



Atividade 2 - Runtime Node.js

Questão 1. Por que `console.log` pode existir no navegador e no Node.js, embora os ambientes sejam diferentes?

O `console.log` existe nos dois ambientes porque tanto o navegador quanto o Node.js disponibilizam uma implementação do objeto `console`. Os dois executam JavaScript, porém cada ambiente fornece APIs próprias: o navegador é voltado à página e à interface, enquanto o Node.js fornece recursos relacionados a arquivos, processos, rede e sistema operacional.

Questão 2. O que `process.version` e `process.platform` representam?

`process.version` informa a versão do Node.js que está sendo utilizada. `process.platform` identifica a plataforma do sistema operacional em que o processo Node.js está executando. Os valores exatos dependem do computador em que os comandos forem executados.

Questão 3. Liste três recursos que fazem sentido no Node.js, mas não são responsabilidades típicas de uma página no navegador.

Acesso ao sistema de arquivos; execução e controle de processos e variáveis de ambiente; e criação de servidores ou comunicação em rede.

Atividade 3 - Variáveis e escopo

Questão 1. Por que usar `const` e `let` no lugar de `var`?

`const` deve ser utilizado quando a variável não precisa ser reatribuída. `let` deve ser utilizado quando o valor precisa mudar durante a execução. Essa escolha torna o código mais previsível e reduz alterações acidentais.

Questão 2. O que acontece ao alterar apenas `service.active` em um objeto declarado com `const`?

A alteração é permitida porque `const` impede a reatribuição da variável, mas não impede a alteração das propriedades internas do objeto.

Questão 3. Por que `const service = {}` não torna o objeto imutável?

Porque `const` protege a referência armazenada na variável, e não o conteúdo interno do objeto. Assim, as propriedades podem ser alteradas, embora a variável não possa receber outro objeto por reatribuição.

Atividade 4 - Tipos e comparações

Questão 1. Qual é a diferença entre `==` e `===`?

O operador `==` compara valores e pode realizar conversão automática de tipos. Já `===` compara valor e tipo ao mesmo tempo, sendo mais seguro e recomendado em código de backend.

Questão 2. Por que a validação de `discountPercentage` precisa verificar `null` e `undefined` explicitamente?

Porque o valor `0` pode ser um desconto válido, mesmo sendo considerado `falsy` em JavaScript. Por isso, `null` e `undefined` devem ser verificados de forma explícita.

Questão 3. O que acontece com `Number("abc")` e `Number.isNaN()`?

`Number("abc")` retorna `NaN` porque o texto não pode ser convertido em número. `Number.isNaN()` pode ser utilizado para verificar se o resultado da conversão é inválido.

Atividade 5 - Strings, template literals e conversões

Questão 1. Qual é a utilidade de template literals?

Template literals permitem inserir variáveis diretamente dentro de uma string usando `${}` , deixando o código mais legível e evitando várias concatenações.

Questão 2. Como converter "19.90" para number e exibir duas casas decimais?

O valor deve ser convertido com `Number()`. Para a apresentação com duas casas decimais, pode-se usar `toFixed(2)`.

Questão 3. Qual é a diferença entre `parseInt("10min", 10)` e `Number("10min")`?

`parseInt("10min", 10)` retorna 10 porque lê a parte numérica no início da string e para quando encontra caracteres inválidos. `Number("10min")` tenta converter todo o conteúdo e retorna NaN porque a string não é totalmente numérica.

Atividade 6 - Operadores modernos

Questão 1. Para que serve optional chaining (?.)?

O optional chaining permite acessar uma propriedade sem gerar erro caso alguma parte do caminho não exista. Se o valor intermediário for nulo ou indefinido, o resultado será undefined.

Questão 2. Qual é a diferença entre `0 || 20` e `0 ?? 20`?

O operador `||` considera 0 como falsy e retorna 20. Já `??` só utiliza o valor alternativo quando o valor da esquerda é null ou undefined, por isso mantém o 0.

Questão 3. Por que if/else pode ser mais legível que um ternário aninhado?

Quando existem várias condições, o if/else deixa cada decisão mais clara, reduz a dificuldade de leitura e facilita a manutenção do código.

Atividade 7 - Controle de fluxo

Questão 1. Qual é a vantagem do early return?

O early return encerra a função assim que uma condição inválida é encontrada, reduzindo níveis de if aninhados e deixando o fluxo principal mais simples.

Questão 2. Quando o switch é útil?

O switch é útil quando existem várias opções bem definidas para o mesmo valor, como os perfis ADMIN, PROFESSIONAL e CUSTOMER.

Questão 3. Por que refatorar três if aninhados usando early return?

A refatoração trata primeiro as condições inválidas e reduz a profundidade do código. Isso melhora a legibilidade e facilita futuras alterações.

Atividade 8 - Funções

Questão 1. Por que calculateEndTime é fácil de testar quando recebe todos os dados por parâmetro?

Porque a função não depende de variáveis externas. Com as mesmas entradas, ela produz o mesmo resultado, tornando seu comportamento previsível.

Questão 2. Por que `formatService` não deve alterar o objeto recebido?

A função deve apenas ler os dados necessários e produzir uma nova string. Evitar alterações no objeto recebido reduz efeitos colaterais e torna o código mais seguro.

Questão 3. Qual das duas funções é mais fácil de testar?

As duas são simples de testar porque recebem os dados por parâmetro e não dependem de estado externo. `calculateEndTime` é ainda mais direta por trabalhar apenas com valores numéricos e retornar o resultado do cálculo.

Atividade 9 - Closures e estado

Questão 1. Como `createSequence` mantém o valor entre chamadas?

A função interna mantém acesso à variável criada no escopo de `createSequence`. Esse comportamento é uma closure e permite preservar o estado entre as chamadas da função retornada.

Questão 2. O que acontece com o estado quando o processo Node.js é encerrado?

Todo estado armazenado somente em memória é perdido. Ao iniciar o processo novamente, esse estado começa do zero, a menos que tenha sido salvo em um meio de persistência externo.

Questão 3. Por que duas instâncias do backend não compartilham esse contador?

Cada processo possui sua própria memória. Portanto, cada instância mantém uma cópia independente do estado. Para compartilhar informações, seria necessário utilizar uma persistência ou serviço externo.

Atividade 10 - Arrays e programação orientada a coleções

Questão 1. Para que serve `filter` no exercício de serviços?

`filter` cria um novo array contendo somente os elementos que atendem à condição definida, como serviços ativos com preço maior que 100.

Questão 2. Para que serve `map` ao gerar apenas id e name?

`map` transforma cada elemento do array e cria uma nova coleção. Nesse caso, cada serviço é convertido para uma representação contendo apenas id e name.

Questão 3. Para que serve `some` ao verificar serviços inativos?

`some` retorna true quando pelo menos um elemento do array atende à condição definida, permitindo verificar rapidamente se existe algum serviço inativo.

Questão 4. Como `reduce` é utilizado no cálculo do preço médio?

`reduce` acumula os preços de todos os serviços em um único total. Depois, o total é dividido pela quantidade de serviços para obter o preço médio.

Desafio profissional - paginação

A função `paginate(items, page, pageSize)` calcula o índice inicial de uma página e utiliza `slice` para retornar somente os itens daquele intervalo. No teste realizado, foram utilizados 25 itens, divididos em páginas de 10 registros.